Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине «Проектирование программного обеспечения интеллектуальных систем»
За 1 семестр
Тема: «Иерархии классов. Наследование»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
Проверил:
Монтик Н.С.

Брест 2026

Цель работы: Изучить правила наследования классов; реализовать множественное наследование; изучить управление методами и свойствами производных классов через объекты и через указатели на объекты.

Ход работы:

Задание: Реализовать иерархию из трёх классов: два базовых класса и производный класс, наследующий оба базовых (множественное наследование). В обоих базовых классах реализовать метод с одинаковым именем и разным поведением; в производном классе создать метод, вызывающий оба варианта с помощью разрешения области видимости. Во всех классах определить конструкторы и переменные-члены. Отдельно реализовать пример с виртуальными функциями: в одном из базовых классов объявить метод `virtual`, в производном переопределить его с ключевым словом `override`. В `main()` продемонстрировать вызовы через объекты и через указатели (`->`), а для виртуального метода - различие вызова через указатель базового класса и через объект.

Описание иерархии классов:

Иерархия построена в предметной области «Студент»: производный класс WorkingStudent (студент-совместитель) наследует `public` оба базовых класса - Person (человек) и Worker (работник).

1. Базовый класс Person.

- поля: `name` - имя, `age` - возраст;
- `Person(const string& n, int a)` - конструктор; `virtual ~Person()` - виртуальный деструктор;
- `TO_GET_Name()`, `TO_GET_Age()` - методы получения значений;
- `void TO_INTRODUCE() const` - невиртуальный метод (будет иметь такое же имя в Worker);
- `virtual void TO_DESCRIBE() const` - виртуальный метод для демонстрации позднего связывания.

2. Базовый класс Worker.

- поля: `company` - место работы, `hours` - часов в неделю;

- `Worker(const string& c, int h)` - конструктор; `virtual ~Worker()` - виртуальный деструктор (нужен, так как объект можно удалить и через `Worker*`);
- `TO_GET_Company()`, `TO_GET_Hours()`, `TO_SET_Hours(int h)` - методы доступа;
- `void TO_INTRODUCE() const` - метод с тем же именем, что и в Person, но с другим поведением.

3. Производный класс WorkingStudent.

- `class WorkingStudent : public Person, public Worker` - множественное наследование; поля: `course` - курс, `group` - группа;
- конструктор передаёт аргументы конструкторам базовых классов в списке инициализации: `Person(n, a), Worker(c, h)`. Конструкторы базовых классов вызываются в порядке их перечисления в объявлении класса (Person, затем Worker), деструкторы - в обратном порядке;
- `TO_INTRODUCE()` - метод с тем же именем, который скрывает оба базовых и вызывает их явно через оператор разрешения области видимости: `Person::TO_INTRODUCE(); Worker::TO_INTRODUCE();;`
- `void TO_DESCRIBE() const override` - переопределение виртуального метода Person; `override` заставляет компилятор проверить, что в базовом классе есть метод с такой же сигнатурой.

Особенности наследования.

Если бы в WorkingStudent метод `TO_INTRODUCE` не был определён, вызов `ws.TO_INTRODUCE()` оказался бы неоднозначным. Пример (классы Base1 и Base2 с методом `action`):

```
class Derived : public Base1, public Base2 {};
Derived d;
d.action();
```

Ошибка компилятора:

```
error: request for member 'action' is ambiguous
```

```
note: candidates are: 'void Base2::action()'
note:                 'void Base1::action()'
```

Разрешить неоднозначность можно явным указанием класса: `ws.Person::TO_INTRODUCE()`. Невиртуальный метод вызывается по типу указателя: `Person* pp = &ws; pp->TO_INTRODUCE();` вызовет версию Person, а виртуальный `pp->TO_DESCRIBE()` - версию WorkingStudent. При копировании в объект базового типа (`Person copy = ws;`) часть производного класса теряется (срезка), поэтому у `copy` вызывается версия Person. Виртуальный деструктор гарантирует, что при `delete` через `Person*` объект уничтожается полностью.

4. Демонстрация работы.

В `main()` последовательно показано: вызов public-членов базовых классов через объекты `p` и `w`; вызов унаследованных и собственных членов через объект `ws`, включая явный выбор версии `ws.Person::TO_INTRODUCE()`; вызов тех же членов через указатели `WorkingStudent*`, `Person*` и `Worker*` с помощью оператора `->` (через указатель базового класса доступны только члены базового класса); сравнение вызова виртуального и невиртуального методов, срезка и удаление динамического объекта через указатель базового класса.

Код программы:

```
#include <iostream>
#include <string>
#include <cstdlib>
#include <clocale>
using namespace std;

class Person
{
private:
    string name;
    int age;

public:
    Person(const string &n, int a) : name(n), age(a)
    {
        cout << "  [Person] конструктор: " << name << endl;
    }
```

```
    virtual ~Person()
    {
        cout << "  [Person] деструктор: " << name << endl;
    }

    string TO_GET_Name() const { return name; }
    int TO_GET_Age() const { return age; }

    void TO_INTRODUCE() const
    {
        cout << "  Person::TO_INTRODUCE: " << name
             << ", возраст " << age << endl;
    }

    virtual void TO_DESCRIBE() const
    {
        cout << "  Person::TO_DESCRIBE: человек "
             << name << endl;
    }
};

class Worker
{
private:
    string company;
    int hours;

public:
    Worker(const string &c, int h) : company(c), hours(h)
    {
        cout << "  [Worker] конструктор: " << company << endl;
    }

    virtual ~Worker()
    {
        cout << "  [Worker] деструктор: " << company << endl;
    }

    string TO_GET_Company() const { return company; }
    int TO_GET_Hours() const { return hours; }
    void TO_SET_Hours(int h) { hours = h; }

    void TO_INTRODUCE() const
    {
        cout << "  Worker::TO_INTRODUCE: работает в " << company
             << ", " << hours << " ч/нед" << endl;
    }
};

class WorkingStudent : public Person, public Worker
{
private:
```

```
    int course;
    int group;

public:
    WorkingStudent(const string &n, int a, const string &c,
                   int h, int cr, int g)
        : Person(n, a), Worker(c, h), course(cr), group(g)
    {
        cout << "  [WorkingStudent] конструктор: курс "
             << course << endl;
    }

    ~WorkingStudent() override
    {
        cout << "  [WorkingStudent] деструктор: курс "
             << course << endl;
    }

    int TO_GET_Course() const { return course; }
    int TO_GET_Group() const { return group; }

    void TO_INTRODUCE() const
    {
        Person::TO_INTRODUCE();
        Worker::TO_INTRODUCE();
        cout << "  WorkingStudent::TO_INTRODUCE: курс "
             << course << ", группа " << group << endl;
    }

    void TO_DESCRIBE() const override
    {
        cout << "  WorkingStudent::TO_DESCRIBE: студент "
             << TO_GET_Name() << ", курс " << course
             << ", работает в " << TO_GET_Company() << endl;
    }
};

int main()
{
    system("chcp 65001");
    setlocale(LC_ALL, "ru_RU.UTF-8");

    cout << "=== 1. Объекты базовых классов ===" << endl;
    Person p("Иван", 20);
    Worker w("Типография", 12);
    cout << "Person: имя " << p.TO_GET_Name()
         << ", возраст " << p.TO_GET_Age() << endl;
    p.TO_INTRODUCE();
    p.TO_DESCRIBE();
    cout << "Worker: место работы " << w.TO_GET_Company()
         << ", часов " << w.TO_GET_Hours() << endl;
    w.TO_SET_Hours(16);
    cout << "После TO_SET_Hours(16): "
```

```
     << w.TO_GET_Hours() << endl;
w.TO_INTRODUCE();

cout << endl
     << "=== 2. Объект производного класса ===" << endl;
WorkingStudent ws("Юрий", 19, "Фирма", 15, 2, 29);

cout << "Имя: " << ws.TO_GET_Name()
     << ", возраст: " << ws.TO_GET_Age() << endl;
cout << "Работа: " << ws.TO_GET_Company()
     << ", часов: " << ws.TO_GET_Hours() << endl;

cout << "Курс: " << ws.TO_GET_Course()
     << ", группа: " << ws.TO_GET_Group() << endl;
cout << "TO_INTRODUCE производного класса:" << endl;
ws.TO_INTRODUCE();

cout << "Явный вызов ws.Person::TO_INTRODUCE():" << endl;
ws.Person::TO_INTRODUCE();
cout << "Явный вызов ws.Worker::TO_INTRODUCE():" << endl;
ws.Worker::TO_INTRODUCE();
cout << "Виртуальный ws.TO_DESCRIBE():" << endl;
ws.TO_DESCRIBE();

cout << endl
     << "=== 3. Вызов через указатели ===" << endl;
WorkingStudent *pd = &ws;
Person *pp = &ws;
Worker *pw = &ws;

cout << "Через WorkingStudent*:" << endl;
cout << "  " << pd->TO_GET_Name() << ", "
     << pd->TO_GET_Age() << ", "
     << pd->TO_GET_Company() << ", "
     << pd->TO_GET_Hours() << ", "
     << pd->TO_GET_Course() << ", "
     << pd->TO_GET_Group() << endl;
pd->TO_INTRODUCE();

cout << "Через Person*:" << endl;
cout << "  " << pp->TO_GET_Name() << ", "
     << pp->TO_GET_Age() << endl;
pp->TO_INTRODUCE();

cout << "Через Worker*:" << endl;
cout << "  " << pw->TO_GET_Company() << ", "
     << pw->TO_GET_Hours() << endl;
pw->TO_INTRODUCE();

cout << endl
     << "=== 4. Виртуальные функции ===" << endl;
cout << "Прямой вызов ws.TO_DESCRIBE():" << endl;
ws.TO_DESCRIBE();
```

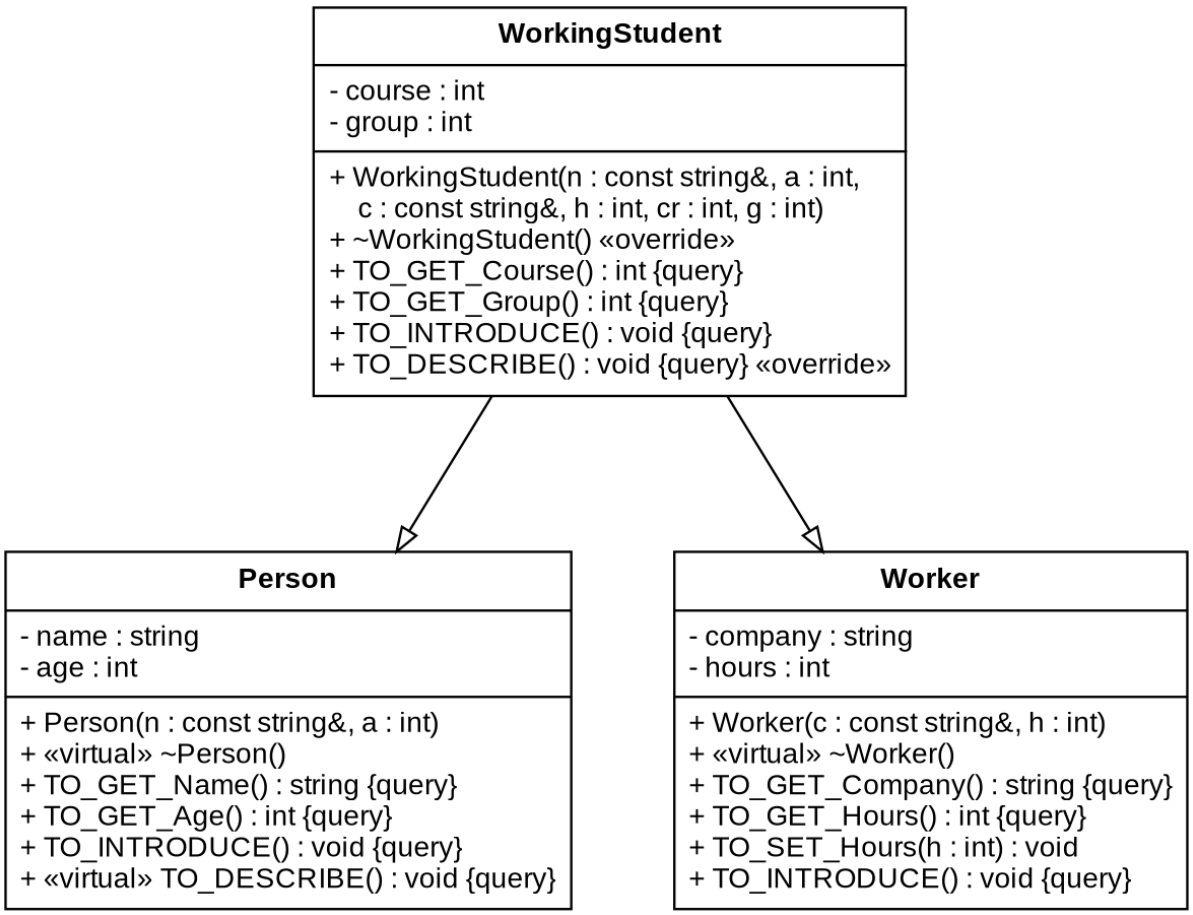
```
    cout << "Через Person* на WorkingStudent (virtual):"
         << endl;
    pp->TO_DESCRIBE();
    cout << "Невиртуальный pp->TO_INTRODUCE() для сравнения:"
         << endl;
    pp->TO_INTRODUCE();

    cout << "Срезка: Person copy = ws; copy.TO_DESCRIBE():"
         << endl;
    {

        Person copy = ws;
        copy.TO_DESCRIBE();
    }
    cout << "Динамический объект через Person*:" << endl;
    Person *dp =
        new WorkingStudent("Анна", 21, "Кафе", 10, 3, 29);
    dp->TO_DESCRIBE();
    cout << "delete dp (виртуальный деструктор):" << endl;
    delete dp;

    cout << endl
         << "Выход из main:" << endl;
    return 0;
}
```

UML-диаграмма классов:



Знак – обозначает закрытые, + - открытые члены. Стрелки с пустым треугольным наконечником показывают наследование: класс WorkingStudent является потомком классов Person и Worker. Стереотипы «virtual» и «override» отмечают виртуальные методы и их переопределение.

Результат:

1. Вызовы через объекты базовых классов:

```
=== 1. Объекты базовых классов ===
  [Person] конструктор: Иван
  [Worker] конструктор: Типография
Person: имя Иван, возраст 20
  Person::TO_INTRODUCE: Иван, возраст 20
  Person::TO_DESCRIBE: человек Иван
Worker: место работы Типография, часов 12
После TO_SET_Hours(16): 16
  Worker::TO_INTRODUCE: работает в Типография, 16 ч/нед
```

2. Вызовы через объект производного класса:

```
=== 2. Объект производного класса ===
  [Person] конструктор: Юрий
  [Worker] конструктор: Фирма
  [WorkingStudent] конструктор: курс 2
Имя: Юрий, возраст: 19
Работа: Фирма, часов: 15
Курс: 2, группа: 29
TO_INTRODUCE производного класса:
  Person::TO_INTRODUCE: Юрий, возраст 19
  Worker::TO_INTRODUCE: работает в Фирма, 15 ч/нед
  WorkingStudent::TO_INTRODUCE: курс 2, группа 29
Явный вызов ws.Person::TO_INTRODUCE():
  Person::TO_INTRODUCE: Юрий, возраст 19
Явный вызов ws.Worker::TO_INTRODUCE():
  Worker::TO_INTRODUCE: работает в Фирма, 15 ч/нед
Виртуальный ws.TO_DESCRIBE():
  WorkingStudent::TO_DESCRIBE: студент Юрий, курс 2, работает в Фирма
```

3. Вызовы через указатели:

```
=== 3. Вызов через указатели ===
Через WorkingStudent*:
  Юрий, 19, Фирма, 15, 2, 29
  Person::TO_INTRODUCE: Юрий, возраст 19
  Worker::TO_INTRODUCE: работает в Фирма, 15 ч/нед
  WorkingStudent::TO_INTRODUCE: курс 2, группа 29
Через Person*:
  Юрий, 19
  Person::TO_INTRODUCE: Юрий, возраст 19
Через Worker*:
  Фирма, 15
  Worker::TO_INTRODUCE: работает в Фирма, 15 ч/нед
```

4. Виртуальные функции, виртуальный деструктор и выход из main():

```
=== 4. Виртуальные функции ===
Прямой вызов ws.TO_DESCRIBE():
  WorkingStudent::TO_DESCRIBE: студент Юрий, курс 2, работает в Фирма
Через Person* на WorkingStudent (virtual):
  WorkingStudent::TO_DESCRIBE: студент Юрий, курс 2, работает в Фирма
Невиртуальный pp->TO_INTRODUCE() для сравнения:
  Person::TO_INTRODUCE: Юрий, возраст 19
Срезка: Person copy = ws; copy.TO_DESCRIBE():
  Person::TO_DESCRIBE: человек Юрий
  [Person] деструктор: Юрий
Динамический объект через Person*:
  [Person] конструктор: Анна
  [Worker] конструктор: Кафе
  [WorkingStudent] конструктор: курс 3
  WorkingStudent::TO_DESCRIBE: студент Анна, курс 3, работает в Кафе
delete dp (виртуальный деструктор):
  [WorkingStudent] деструктор: курс 3
  [Worker] деструктор: Кафе
  [Person] деструктор: Анна

Выход из main:
  [WorkingStudent] деструктор: курс 2
  [Worker] деструктор: Фирма
  [Person] деструктор: Юрий
  [Worker] деструктор: Типография
  [Person] деструктор: Иван
```

Вывод: Изучили наследование классов и реализовали иерархию из двух базовых классов (Person, Worker) и производного класса WorkingStudent с множественным наследованием. Показали, что одноимённые методы базовых классов приводят к неоднозначности, которая устраняется переопределением метода в производном классе и явным указанием области видимости (`Base::method()`). Выяснили, что конструкторы базовых классов вызываются в порядке их перечисления в объявлении, а деструкторы - в обратном порядке. Через указатель базового класса доступны только члены базового класса; невиртуальные методы вызываются по типу указателя, а виртуальные - по типу объекта (`virtual` и `override`). Показали необходимость виртуального деструктора при удалении объекта через указатель на базовый класс и срезку объекта при копировании в базовый тип. Построили UML-диаграмму иерархии классов.